Some Empirical Observations from List-Decoding Interleaved Reed Solomon Codes

Gabrielle Beck

September 5, 2026

1 Introduction

It is well understood that Reed Solomon codes can be decoded up to the Johnson Bound using methods first introduced by Guruswami and Sudan. This algorithm follows a two step approach: first a bivariate polynomial $Q(x, y)$ is constructed and then factored. Because of the structure of the polynomial and its degree bounds, we are guaranteed that if there is a polynomial $p(x)$ going through at least t input points, then $y - p(x)|Q(x, y)$ or equivalently $Q(x, p(x)) = 0$. Cohn and Heninger in [5] showed how it is possible to find Q by using lattices instead of solving linear constraints. Because each vector in the lattice satisfies the constraints that $Q(\alpha, \beta) = 0$ and we can guarantee we will find a short enough Q due to bounds on the shortest vector in the lattice, this method produces results that are equivalent to the original algorithm of Guruswami Sudan.

A separate line of research has considered decoding of Reed Solomon based on Gao's key equation. It is known that the basic version of this algorithm is capable of unique decoding (up to half the minimum distance). Subsequent work has looked at "powering" these equations and then doing so with multiplicity, showing that in practice this approach can correct up to the same number of errors as GS, except that it may fail for a few error patterns past the minimum distance bound [9]. Commom consensis it that this approach is considerably more limited than the first.

In what follows, we provide evidence that this is perhaps an incorrect interpretation. First, note that there is an explicit connection that exists between the lattice decoders for list decoding and power decoding. The decoding equations of [9] produces a lattice which has as its scaled dual the lattice in [5].

It is also the case that the approach of finding the multivariate polynomial Q and then doing a root finding step is theoretically more costly than simply searching the solution space directly.

In the rest of this document we focus specifically on using the above approaches with interleaved reed solomon codes. We give a list decoding algorithm for these codes starting from the approach of [4] which works well heuristically given certain constraints over a limited type of error channel.

2 Preliminaries

In this section we introduce notation and terminology that will be used throughout the rest of the paper. Let $\mathbb{F}$ be a finite field and $\mathbb{F}[z]$ a polynomial ring. We consider a *point* to be a $c+1$ tuple $(\alpha, \beta_1, \ldots, \beta_c) \in \mathbb{F}^{c+1}$ defined over some finite field $\mathbb{F}$. A c tuple of polynomials $P = (p_1, \ldots, p_c) \in \mathbb{F}[z]^c$ *agrees* with a point $(\alpha, \beta_1, \ldots, \beta_c)$ if $\forall j \in [c], p_j(\alpha) = \beta_j$.

A *polynomial lattice* $L(B)$ with basis $B = (\vec{b}_1, \ldots \vec{b}_m) \in \mathbb{F}[z]^{m \times n}$ contains all vectors v that are $\mathbb{F}[z]$-combinations of the basis vectors
i.e. $L(B) := \{\vec{v} \in \mathbb{F}[z]^n : \exists \vec{a} \in \mathbb{F}[z]^m, \vec{v} = \vec{a}B\}$. The length of a vector $\vec{v} = (v_1, \ldots v_n) \in \mathbb{F}[z]^n$ is given by $\mathsf{deg}(\vec{v}) = \mathsf{max}_i \mathsf{deg}(v_i)$. [1] We define rdeg for both vectors and matrices. For $\vec{v} \in \mathbb{F}[z]^n$, $\mathsf{rdeg}(\vec{v}) = (\mathsf{deg}(v_1), \ldots \mathsf{deg}(v_n))$. For matrices $M \in \mathbb{F}[z]^{m \times n}$ $\mathsf{rdeg}(M)$ is the vector of row degrees of M or

$$\mathsf{rdeg}(M) = \mathsf{rdeg}\left(\begin{pmatrix} - & \vec{v}_1 & - \\ & \vdots & \\ - & \vec{v}_m & - \end{pmatrix}\right) = (\mathsf{deg}(\vec{v}_1), \ldots, \mathsf{deg}(\vec{v}_m))$$

In writing, we may also refer to $\mathsf{rdeg}(M)$ as the *row profile* of M.

For a matrix $M \in \mathbb{F}[z]^{m \times n}$, $M_{I,J}$ with $I \subset [m], J \subset [n]$ denotes the $|I| \times |J|$ sub-matrix of M that is constructed by taking rows $i \in I$ and columns $j \in J$. $M_{*,J}$ denotes taking all rows and only the columns in J (and the opposite for $M_{J,*}$). For a vector $\vec{v} \in \mathbb{F}[z]^n$, $v_I \in \mathbb{F}[z]^{|I|}$ for $I \subset [n]$ denotes the vector formed by taking only entries at index I from $\vec{v}$. The *pivot index* of a vector $\vec{v}$ denotes the rightmost index $i \in [n]$ such that $\mathsf{deg}(v_i) = \mathsf{deg}(\vec{v})$. The *pivot set* $I = \{i_1, \ldots i_m\}$ of a matrix $M \in \mathbb{F}[z]^{m \times n}$ is formed by taking the pivot index of each row i.e. for each $j \in [m]$, $\mathsf{deg}(M_{j,\{i_j\}}) = \mathsf{deg}(M_{j,*})$ and $\forall i > i_j, \mathsf{deg}(M_{j,\{i\}}) < \mathsf{deg}(M_{j,*})$.

Let

$$z^{(x_1, \ldots x_n)} = \begin{bmatrix} z^{x_1} & & \\ & \ddots & \\ & & z^{x_n} \end{bmatrix} = Diag(z^{x_1}, \ldots, z^{x_n})$$

$LC(M)$ for a matrix $M \in \mathbb{F}[z]^{m \times n}$ is called the *leading coefficient matrix* of M and entry $\ell_{i,j}$ of $LC(M)$ is the coefficient of $m_{i,j}$ associated with $\deg(\vec{v}_i)$ (possibly 0) where M is composed of the row vectors $\vec{v}_1, \ldots \vec{v}_m$.

A matrix $B \in \mathbb{F}[z]^{m \times n}$ is $\vec{a}$ row-reduced of degree $\vec{\alpha}$ with pivot set I if

$$z^{-\vec{a}_I - \vec{\alpha}} B z^{\vec{a}} = B' + O(z^{-1})_{z \to \infty} \tag{1}$$

where $B' \in \mathbb{F}^{m \times n}$ has full row rank. This is equivalent to saying that $LC(B)$ is non-singular. If each index in the pivot set I is unique then we say B is in *weak* popov form.

[1] For an element $v \in \mathbb{F}[z]$, we define $\mathsf{deg}(v)$ in the standard way as the maximal $i \in \mathbb{N}$ for which the monomial z^i is non-zero. If $v = 0$, then by convention we define $\mathsf{deg}(v) = -1$.

Finding an $\vec{a}$ row-reduced weak popov form of a matrix M can be done efficiently in polynomial time [7]. When we need to do this we will call the routine $\mathsf{WeakPopov}(M, \vec{a})$. From equation (1) we can see that

$$\begin{aligned}
&\mathsf{deg}(\mathsf{det}(z^{-\vec{a}_I - \vec{\alpha}} B_{*,I} z^{\vec{a}_I})) = \mathsf{deg}(\mathsf{det}(B'_{*,I} + O(z^{-1})_{z\to\infty})) \\
&- \sum_i \alpha_i + \mathsf{deg}(\mathsf{det}(B_{*,I})) = 0 \\
&\mathsf{deg}(\mathsf{det}(B_{*,I})) = \sum_i \alpha_i
\end{aligned}$$

Note that in the case where $a = \vec{0}$ and B is square, a reduced basis will always find at least one vector $\vec{v}$ s.t.

$$\deg(\vec{v}) \leq \frac{\deg(\mathsf{det}(L(B))}{\dim(L)}$$

Lemma 1 (Predictable Degree Property)**.** *Let $B \in \mathbb{F}[z]^{m\times n}$ be $\vec{c}$ row-reduced with $\vec{\beta} = \mathsf{rdeg}(Bz^c)$ and suppose $C = PB$ where $\vec{\gamma} = \mathsf{rdeg}(Cz^c)$. Then $\deg(P_{i,j}) \leq \vec{\gamma}_i - \vec{\beta}_j$*

As mentioned in [8] for any matrix $M \in \mathbb{F}[z]^{m\times n}$ we also have the additional property that for any $\vec{s} \in \mathbb{F}(x)^m$ s.t. $\vec{s}M \in \mathbb{F}[x]^n \setminus \{\vec{0}\}$ the pivot index of $\vec{s}M$ is in the pivot set of any weak popov reduced form of M.

The next theorem gives an upper degree bound on U where $T = UA$ and T is an $\vec{a}$ row reduced matrix with pivot set I and degree $\vec{\alpha}$. It has been adapted from [1] mainly to use row reduction instead of column reduction and for the special setting of $\vec{c} = \vec{a}$.

Theorem 2 (Degree bounds for Multiplier U)**.** *Let $A = UT$ where $A, T \in \mathbb{F}[z]^{m\times n}, U \in \mathbb{F}[z]^{m\times m}$ and T be $\vec{a}$-row reduced of degree $\vec{\alpha}$ with pivot set I. Define*

$$\begin{aligned}
&\vec{\tau} \leftarrow \mathsf{rdeg}(Tz^{\vec{a}}), \vec{\gamma} \leftarrow \mathsf{rdeg}(Az^{\vec{a}}) \\
&\vec{\Delta}^{\vec{a}} = |\vec{\gamma}| - |\vec{a}_I| - |\vec{\alpha}|
\end{aligned}$$

Then we have the following bounds on the entries of U

$$\deg(U)_{i,j} \leq \vec{\tau}_i - \vec{\gamma}_j + \vec{\Delta}^{\vec{a}} \tag{2}$$

$\vec{\Delta}^{\vec{a}} = 0$ if and only if A is also $\vec{a}$-row reduced. Moreover,

$$\vec{\tau} = \vec{\alpha} + \vec{a}_I \leq \mathrm{perm}(\vec{\gamma}) \tag{3}$$

Interleaved Reed Solomon Codes Roughly speaking, an interleaved reed solomon code in our work will be a code $C : (\mathbb{F}^c)^{k+1} \to (\mathbb{F}^c)^n$ defined by the mapping of c k degree polynomials $p_1 \dots p_c$ to $\{(p_1(\alpha_i), \dots, p_c(\alpha_i))\}_{i\in[n]}$ where the alphabet is $\mathbb{F}^c$, the block length of the code is n and the dimension is $k+1$.[2] We choose the input points at random i.e. $\alpha_1, \dots \alpha_n \xleftarrow{\$} \mathbb{F}$ and also assume that the field is exponentially sized i.e. $n \ll |\mathbb{F}|$. The α_i values are assumed to be part of the description of the code C and thus do not need to be sent over the wire. That said, we may abuse notation in this work and treat it as part of the input in some of our problem statements. In practice some relevant applications may require that these values be sent, in which case the results below still follow provided that the α_i values do not collide.

Channel Setting. We consider a subclass Ch of the class of channels $\mathcal{C}$ where $Ch \in \mathcal{C}$ is a function $Ch : (\mathbb{F}^c)^n \to (\mathbb{F}^c)^n$. Each $Ch \in \mathsf{Ch}$ can be associated with a list of sets $M = [S_1, \dots, S_z]$ where $\forall i \in [z]$, $S_i \subseteq PS([n])$ where $PS([n])$ denotes the power set of $[n]$. We write Ch^M to classify such channels and give their description below:

$Ch^M(\{(\beta_{i,1}, \dots \beta_{i,c})\}_{i\in[n]}) :$
  $I' :=(\{(\beta_{i,1}, \dots \beta_{i,c})\}_{i\in[n]}$
  $\forall S_i \in M,$
    $p_1 \dots p_c \xleftarrow{\$} (\mathbb{F}^{k+1})^c$
    $\forall s_{i,j} \in S_i,$ replace value at index $s_{i,j}$ in I'
     with $(p_1(\alpha_{s_{i,j}}), \dots p_c(\alpha_{s_{i,j}}))$
  return I'

At a high level, the channel is equivalent to doing the following: for each set $S_i \in M$ sample a random message m' for the IRS code C, and for each $s_{i,j} \in S_i$ replace the $s^{th}_{i,j}$ symbol of the original codeword with the $s^{th}_{i,j}$ symbol of $C(m')$. Such a channel induces e errors if e of the original symbols have been replaced i.e. $w(c, Ch^m(c)) = e$ where w is the hamming weight.

We acknowledge that the channel description above is non-standard. However, it models at least one real use-case: recovering a dealer's input in the multi-dealer secret sharing of [6] is more or less equivalent to list decoding over this channel. This class of channels is also non-trivial. Assuming $\exists i$ s.t. $|S_i| > k$ it is not simulatable by a random error channel. On the other hand, it is quite weak in comparison to a truly adversarial channel: the degree of the input polynomials cannot be above k and the messages are chosen uniformly at random.

The main problem we will consider in this channel setting is list-decoding, where it is only important to recover a list of all closest codewords rather than

[2] The more natural decision is to make k the dimension but this would mean the polynomials are bound by degree $k-1$ rather than k

the original message that was sent. Tailored to our use of interleaved reed solomon codes, we re-phrase this question below.

List-Decoding Problem for IRS Codes. Given a list of input points $\{(\alpha_i, \beta_{i,1}, \ldots \beta_{i,c})\}_{i \in [n]}$, a degree bound k, and an error parameter e, recover all tuples $P = (p_1, \ldots p_c) \in \mathbb{F}[z]$ where $\forall i \in [c], \mathsf{deg}(p_i) \leq k$ and $\exists T \subseteq [n]$,s.t. $|T| \geq n - e$ and $\forall i \in [c], \forall j \in T, p_i(\alpha_j) = \beta_{j,i}$. For a given code C, a list of input points I and error parameter e we denote the set of all tuples that generate codewords within hamming distance e of I by $\mathsf{HamRadius}_C(I, e)$.

Our general heuristic result (stated informally) is given below.

Heuristic 3. *For all choices of IRS codes C, there exists a polynomial time algorithm A s.t. $\forall Ch^M \in \mathsf{Ch}$ where Ch^M induces at most $e \leq \frac{c}{c+1}(n - (k+1))$ errors, $\forall m \in (\mathbb{F}^c)^{k+1}, \mathsf{Pr}\Big[\mathsf{A}(Ch^M(C(m))) \neq \mathsf{HamRadius}_C(I, e)\Big] \leq \mathsf{negl}(n)$ where the probability is taken over the random coins used in Ch^M, n is the block length of C, $k+1$ is its dimension, and the alphabet of C is $\mathbb{F}^c$.*

To build to this result we first look at an easier channel setting, where instead of choosing polynomials $p_1, \ldots p_c$ at random such that $\mathsf{deg}(p_i) \leq k$ we consider the case where for all p_i the coefficient associated with the monomial x^k is random, conditioned on being non-zero. Call this more limited class of channels Ch'. In this case, we can give a proof, although it relies on an assumption on the behavior of the decoding lattice. We have evidence that this assumption does not hold in general, as described at the end of this document, particularly in the case of small fields. But even in those sub-cases there appear to workarounds that still allow for successful list-decoding.

Theorem 4. *For any choices of IRS codes C, there exists a polynomial time algorithm A s.t. $\forall Ch^M \in \mathsf{Ch}'$ where Ch^M induces at most $e \leq \frac{c}{c+1}(n - (k+1))$ errors, $\forall m \in (\mathbb{F}^c)^k, \mathsf{Pr}\Big[\mathsf{A}(Ch^M(C(m))) \neq \mathsf{HamRadius}_C(I, e)\Big] \leq \mathsf{negl}(n)$ where the probability is taken over the random coins used in Ch^M, n is the block length of C, $k+1$ is its dimension, and the alphabet of C is $\mathbb{F}^c$ provided Assumption 5 holds.*

3 Warm-up: Berlekamp Welch

One way to use lattices to do Reed Solomon decoding follows the framework of Berlekamp-Welch. In this decoding method, we focus on the error polynomial $e(z)$ of a polynomial $p(z)$ that should be recovered. If $f(z)$ is the lagrangian interpolation of our input points, then we know $p(z)e(z) = f(z)e(z)$ on all input points. This leads to a linear system we can solve over $\mathbb{F}$ if we replace $p(z)e(z)$ with another variable $q(z)$ and then jointly solve for the coefficients of $e(z)$ and $q(z)$, using constraints built from $q(\alpha_i) - f(\alpha_i)e(\alpha_i) = 0$ for each point α_i. The final target solution is then recovered by computing $q(z)/e(z)$. Note that we

require our linearized solution to satisfy a non-trivial division property, but in the special case where e is less than the unique decoding bound, it is easy to satisfy.

However, this is not the only way we can view this problem. Instead of solving a linear system over $\mathbb{F}$, consider the set of all $a(z), b(z) \in \mathbb{F}[z]$ that satisfy $a(z) - b(z)f(z) = 0 \pmod{N(z)}$ where $N(z)$ is the minimal polynomial with 0's at all input points. Note that this set is actually a lattice L over $\mathbb{F}[z]^2$: if $a(z), b(z)$ satisfies the equation and so does $c(z), d(z)$, then $m_1(z)a(z) + m_2(z)c(z), m_1(z)b(z) + m_2(z)d(z)$ also satisfies the equation for any $m_1, m_2 \in \mathbb{F}[z]$. A short vector in L itself is not equivalent to the q and e values we found earlier. In particular, in that setting we had the additional constraint that $\deg(q) = \deg(e) + \deg(p)$. We simulate this same gap by finding a shortest vector in the lattice formed by shifting up the degree of the entry in the column associated with a by $\deg(p)$.

For correctness of the lattice technique in the unique decoding setting, we must consider the length of the *target vector*, that is the vector that corresponds to the solution we wish to recover. For our toy example, this is the vector $\vec{t} = [z^{\deg(p)}e(z), e(z)p(z)]$. Given a description of a basis for the lattice L we can calculate $\det(L)$ and thus give an upper bound on the shortest vector length. It is here that we make the assumption that if we set parameters so that $\deg(\vec{t})$ is less than the shortest vector length then we will recover $\vec{t}$ in any short basis of L. In other words, there are no other unusually short vectors in the lattice.

In this work, we consider lattices that contain *multiple* target vectors (and possibly *near target* vectors)[3] that can be recovered. Our assumption at a high level is that the shortest vectors in any reduced basis of L are $\mathbb{F}[z]$ combinations of target vectors or near target vectors. Note that this is a very natural extension of the prior assumption.

Assumption 5. *Suppose L is the decoding lattice given in the following algorithms and $\vec{t}_1, \ldots \vec{t}_m$ is a list of all target and near target vectors in L. Let B be a reduced basis of L with row vectors $\vec{b}_1, \ldots \vec{b}_n$. For any $j \in [n]$ such that $\deg \vec{b}_j \leq \frac{\deg(\det(L))}{\dim(L)}$, $\vec{b}_j$ can be written as an $\mathbb{F}[z]$ combination of $\vec{t}_1, \ldots, \vec{t}_m$.*

4 Extending to IRS

To provide a structured argument for Heuristic G: , we must first describe the decoder. Consider again the second decoding strategy described in the previous section: we defined a lattice that represented all vectors $\vec{v} = [a(z), b(z)]$ that correspond to solutions to the equation $a(z) - b(z)f(z) = 0 \pmod{N(z)}$ found a shortest vector in this lattice $[a^*(z), b^*(z)]$, and mentioned that a polynomial solution could be given by $b^*(z)/a^*(z)$, provided that the number of errors is less than the unique decoding bound, i.e. $e < \frac{n-k+1}{2}$.

[3] A *near target* vector is a vector $(e(z), p_1(z)e(z), \ldots p_c(z)e(z))$ in L that would be a target vector except $\deg(e(z))$ is slightly larger than e and not as large as $n - k$.

Suppose we instead have an interleaved code where the code consists of symbols of the form $(\alpha, p_1(\alpha), \ldots, p_c(\alpha))$. Just like before, we can construct a lattice for which a short vector will correspond to solutions that we satisfy a system of polynomial equations. For each $i \in [c]$, we construct the key equation of the first part where $a(z) - b_i(z)f_i(z) = 0 \pmod{N(z)}$ and f_i is the lagrangian interpolation of the input on the first coordinate and the $i+1^{th}$ coordinate. We then define a lattice containing all vectors $[a(z), b_1(z), \ldots, b_c(z)] \in \mathbb{F}[z]^{c+1}$ that satisfy this equation. The full algorithm is given below as Algorithm 1. The hope is that - if we set k to be an upper bound on the size of $\deg(p_i(z))$ - then a short basis of the shifted lattice $Lz^{(k,0\ldots,0)}$ will contain our target vector $\vec{t} = [z^k e(z), p_1(z)e(z), \ldots, p_c(z)e(z)]$. To identify the total number of errors we can correct from, we analyze the bound needed on e to ensure $\deg(\vec{t})$ is smaller than the upper bound on the shortest vector in $Lz^{(k,0,\ldots,0)}$ and see this is true when $e < \frac{c}{c+1}(n-(k+1))$. This decoding strategy is well known in the literature and its variants have been shown to satisfy correctness over a random error channel where the field size is exponential in n [2].

Algorithm 1: Lattice-Based IRS Decoder

Input : $k, t, n, \mathsf{In} = \{(\alpha_i, \beta_{i,1}, \ldots, \beta_{i,c})\}_{i=1}^{n}$
Output: $(p_1, \ldots, p_c)$ or $\perp$

$\forall j \in [c], f_j(z) = \mathsf{LagrInterpol}(\{(\alpha_i, \beta_{i,j})\}_{i\in[n]})$, $N(z) = \prod_{i\in[n]}(z - \alpha_i)$

$$M = \begin{bmatrix} 1 & f_1(z) & f_2(z) & \ldots & f_c(z) \\ & N(z) & & & \\ & & N(z) & & \\ & & & \ddots & \\ & & & & N(z) \end{bmatrix}$$

$M_{\mathsf{red}} \leftarrow \mathsf{WeakPopov}(M, (k, 0, \ldots, 0))$

Look for a vector $\vec{v}$ of form $(z^k e(z), p_1(z)e(z), \ldots, p_c(z)e(z))$ in M_{red}
return $(p_1, \ldots p_c)$ *or* $\perp$

While the previous algorithm is sufficient for random errors it is not immediately clear how effective it can be when given more structured input. For one example, consider a case where the input produces a lattice with two vectors we must recover, denoted $\vec{t_1}$ and $\vec{t_2}$. Any $\mathbb{F}[z]$ combination of these vectors is also in L, including something like $\vec{t_1} + \vec{t_2}$. Moreover, this vector may have the *same length* as the shortest target vectors we want to find. What is the guarantee that an algorithm to find a short basis will not find something like this instead of $\vec{t_1}$ or $\vec{t_2}$?

Unfortunately there is none. Despite this, we will show in this section that in some constrained settings it is still possible - even quite easy - to recover all target vectors.

Lemma 6. *Suppose that we sample* $\forall i, j \in [c], p_j^i \xleftarrow{\$} \mathbb{F}_q[x]$ *s.t.* $\deg p_j^i \leq k$ *and* $\alpha_i \xleftarrow{\$} \mathbb{F}\ \forall i \in [n]$. *Then* $\Pr[\exists h, h' \in [c], j \in [n] s.t., \forall i \in [c], p_i^h(\alpha_j) = p_i^{h'}(\alpha_j)]$ *can be made negligible for large enough* $\mathbb{F}$.

Lemma 7. *Let* $\vec{t}_1, \ldots, \vec{t}_m \in \mathbb{F}[z]^{c+1}$ *be polynomial vectors such that* $\vec{t}_i = (e_i(z), e_i(z)p_{i,1}(z), \ldots, e_i(z)p_{i,c}(z))$, $\deg(p_{i,j}) \leq k$, *where the coefficient associated with* z^k *of* $p_{i,j}$ *is drawn uniformly from* $\mathbb{F}\backslash\{0\}$ *and the lead coefficient of* $e_i(z)$ *is* 1. *Then w.h.p.* $\deg(\sum_i a_i\vec{t}_i) = \max_{i^*} \deg(\vec{t}_{i^*}) + \deg(a_{i^*})$

Proof. We will do a proof of contradiction and show that it is very unlikely that $\deg(\sum_i a_i\vec{t}_i) < \max_{i^*} \deg(\vec{t}_{i^*}) + \deg(a_{i^*})$. It should be obvious that it cannot be greater. Let $\vec{w} = \sum_i a_i\vec{t}_i$. We first note that $\forall i$, the lead coefficients of $p_{i,1}(z)e(z), \ldots, p_{i,c}(z)e(z)$ are uniformly random. Unless $\deg(a_j) = 0$ for all $j \in [m]$, in order that $\deg(\vec{w}) < \max_{i^*} \deg(\vec{t}_{i^*}) + \deg(a_{i^*})$ it must be the case that higher order terms cancel. To be specific, if we let $d^* = \max_{i^*} \deg(\vec{t}_{i^*}) + \deg(a_{i^*})$ then at the very least the coefficient associated with z^{d^*} must be 0 *for every position in* $\vec{w}$. We can translate this requirement into a linear system of equations where we have one constraint for each $i \in [c]$ s.t. $\sum_{j\in[m]} a'_j p'_{j,i} = 0$ and a'_j is the lead coefficient of a_j and $p'_{j,i}$ is the lead coefficient of $p_{j,i}$. Because there are c such conditions and $m \leq c$ according to Lemma 6, we get a matrix $M \in \mathbb{F}^{c\times m}$ that has at least as many rows as columns. Since the entries are randomly chosen from a finite field $\mathbb{F}$ the probability is high that M has a square $m \times m$ sub-matrix that is invertible. Thus the only solution to $M\vec{x} = \vec{0}$ is $\vec{x} = \vec{0}$. □

The previous result is quite powerful. Let $\vec{t}_1 \ldots \vec{t}_m$ be the complete list of target vectors associated with some fixed In produced by a channel in Ch'. We suppose this list is ordered by length with $\lambda_1 = \deg(\vec{t}_1), \ldots, \lambda_m = \deg(\vec{t}_m)$ and let S be the matrix formed by taking $\vec{t}_1, \ldots, \vec{t}_m$ as its row vectors. From Assumption 5, we know that any short vector $\vec{b}$ in a reduced basis B, $\exists \vec{a} \in \mathbb{F}[z]^m$ s.t. $\vec{b} = \vec{a}S$. Suppose $\exists i, j \in [m]$ such that $\lambda_i < \lambda_j$ and $\forall h < i, \lambda_i = \lambda_h$. Lemma 7 ensures that the shortest i vectors in a reduced basis B are entirely supported on $\vec{t}_1, \ldots \vec{t}_i$ i.e. $\vec{b} = (a_1, \ldots a_i, 0, \ldots 0)S$.

When we decode, we do not know S, but only a shortest basis B. However, we may easily identify rows in B that are not supported across all rows of S. Let $A_i(z) = N(z)/e_i(z)$. By Lemma 6, for any $\vec{b}$ with $a_i = 0$, we know that $A_i(z)|\vec{b}$ since $A_i(z)|e_j(z), \forall j \neq i$. Thus, one thing we can do once we find a reduced basis is to identify all points α_i in the original input such that $(z - \alpha_i)|\vec{b}_j$ for some row $\vec{b}_j$ in B. We can then consider our original problem with an input set that has been shrunk so that the shortest vectors in the new lattice L' only correspond to our shortest target vectors, say $\hat{S}$.

For any shortest basis B' of L', we know that $B' = U\hat{S}$ where $B' \in \mathbb{F}^{m'\times(c+1)}$, $m' \leq m$ and $U \in \mathbb{F}[z]^{m'\times m'}$ is unimodular. B' is $(k, 0, \ldots 0)$ row-reduced by definition. $\hat{S}$ is also $(k, 0, \ldots 0)$ row-reduced: the leading coefficient matrix of $\hat{S}z^{(k,0,\ldots,0)}$ has full row rank since it has $\hat{S}_{[m'],\{1,\ldots,c+1\}}$ as a sub-matrix and this matrix has an invertible sub-matrix of size $m' \times m'$. $\mathsf{rdeg}(\hat{S}) = (k+d, \ldots, k+d)$

for some fixed degree d. Because $\hat{S}$ and B' are unimodularly equivalent and row reduced, we know that $\mathsf{rdeg}(B') = \mathsf{perm}(\mathsf{rdeg}(\hat{S})) = \mathsf{rdeg}(\hat{S})$. Combined with the predictable degree property, $\forall i, j \in [m'], \deg(U_{i,j}) = 0$. Therefore, $U \in \mathbb{F}^{m' \times m'}$. With such a constrained U we will now give an algorithm which can correctly identify a single target vector in $\hat{S}$ with high probability. Once this vector has been identified, we can permanently remove all inputs that agree with it from In and simply run the decoder again on a smaller set. As long as we can find a single target vector each time we run the main part of the algorithm, we can iteratively find all solutions.

We now describe an algorithm that can identify a target solution $\vec{t}$ in $\hat{S}$, given a short basis B and that $\exists U \in \mathbb{F}^{m \times m}, B = U\hat{S}$. It is depicted as Algorithm 2.

Algorithm 2: FindTarget

Input : $n, k, t, B \in \mathbb{F}[z]^{m \times m}, \mathsf{In} = \{(\alpha_i, \beta_{i,1}, \ldots, \beta_{i,c})\}_{i=1}^{n}$
Output: $(p_1, \ldots, p_c)$

$b_1, \ldots b_m \leftarrow B_{0,*}$
$\alpha \xleftarrow{\$} \{\alpha_i\}_{i \in [n]}$

$$M' = \begin{bmatrix} b_1 & & \\ \vdots & I_m & \\ b_m & & \\ (z-\alpha) & 0 \ \ldots \ 0 & \end{bmatrix}$$

$M'_{\mathsf{red}} \leftarrow \mathsf{WeakPopov}(M', (1, 0, \ldots, 0))$
Discard rows $\vec{v}$ from M'_{red} with $\deg(\vec{v}) \geq 1$, and then set $W = M'_{\mathsf{red}*,\{1,\ldots,m\}}$
$\vec{r} \xleftarrow{\$} \mathbb{F}^{m-1}$
$(w_0, \ldots w_c) \leftarrow \vec{r}WB$
$Q = (\frac{w_1}{w_0}, \ldots, \frac{w_c}{w_0})$
$E \leftarrow \{i \in [n] : Q(\alpha_i) \neq (\beta_{i,1}, \ldots, \beta_{i,c})\}$
$\forall j \in [c], p_j \leftarrow \mathsf{LagrInterpol}(\{(\alpha_i, \beta_{i,j}\}_{i \in E})$
return $(p_1, \ldots p_c)$

Lemma 8. *Given a matrix $B \in \mathbb{F}^{m \times (c+1)}$ that corresponds to short vectors in the lattice specified by M in Algorithm 1 and that $B = US$ where $U \in \mathbb{F}^{m \times m}$ and each row $\vec{s}_i$ in S is of the form $[z^k e_i(z), e_i(z)p_{i,1}(z), \ldots e_i(z)p_{i,c}(z)]$ and $\mathsf{rdeg}(\vec{s}_i) = \mathsf{rdeg}(\vec{s}_j) \forall i, j \in [m]$ then with high probability Algorithm 2 outputs $p_{j^*,1}(z), \ldots p_{j^*,c}(z)$ for the $j \in [m]$ where In agrees with $(p_{j^*,1}, \ldots p_{j^*,c})$ at input α.*

Proof. By Lemma 6, with high probability the point α chosen in the algorithm only agrees with a single solution and *disagrees* with all other solutions. Or in other words, $(z - \alpha) | \vec{s}$ for all solution vectors *except* the one that corresponds

to $p_{j^*,1}(z), \ldots p_{j^*,c}(z)$. Say it's called $\vec{s}_j^*$. Consider vectors $v(z) \in L(B)$ s.t. $(z-\alpha)|v(z)$. Since each vector $\vec{b}_i \in B$ can be written as a linear combination of vectors in S we have the following:

$$\begin{aligned} v(\alpha) &= \sum_i a_i \vec{b}_i(\alpha) \\ &= \sum_i a_i \Big(\sum_j a'_j \vec{s}_j(\alpha) \Big) \\ &= \sum_j \Big(a'_j \sum_i a_i \Big) \vec{s}_j(\alpha) \\ &= \Big(a'_{j^*} \sum_i a_i \Big) \vec{s_{j^*}} \\ &= \vec{0} \end{aligned}$$

We know for a fact that $\deg(a'_j) = 0$ from the premise. Suppose we could restrict to $a_i \in \mathbb{F}$ also, then the only way for $(z-\alpha)|v$ is if in the coordinates corresponding to the basis S, $\vec{v}$ has a 0 value for the index associated with $\vec{s}_{j^*}$. It should be evident that we can only form $m-1$ such linearly independent vectors with these restrictions. The shortest vectors in $L(M')$ are in exact correspondence with the $(a_1, \ldots a_m) \in \mathbb{F}^m$ described previously: the shortest possible vectors in $L(M')$ are those with $\deg \vec{v} = 0$ which is only possible if the first column is 0. This implies the first position of $\vec{v}$ equals 0 when evaluated on α which also means $(z-\alpha)|\vec{v}$ since all vectors in L satisfy $v_i - v_0 = 0 \pmod{N(z)}$.

The above discussion means that the W we find has dimension $m-1$ and when we write $W = U'S$ where $U' \in \mathbb{F}^{(m-1)\times m}$ it has an all 0 column at j^*. If we take a random linear combination of the rows, with high probability the resulting $\vec{w}$ only has a single 0 at j^*. If we consider the vector of *rational* functions $\frac{w_1}{w_0}, \ldots \frac{w_c}{w_0}$ we can deduce that because of the description of L - and provided α_j is not a pole of w_0 - we have that α_j agrees with the input i.e. $f_i(\alpha_j) = \frac{w_i(\alpha_j)}{w_0(\alpha_j)}$. For the $\vec{w}$ we found earlier, this means the vector of rational functions will agree on all input points except those that agree with $p_{j^*,1}(z), \ldots p_{j^*,c}(z)$. This is the exact set we interpolate so we recover the relevant solution, provided the random combination we chose did not zero out another position in the S coordinates. □

We can obviously increase our probability of success by trying many different linear combinations and we can replace the final interpolation step by factoring the first entry of $\vec{w}$ if desired. We are finally ready to describe the decoder for the channels considered in Theorem 4. A formal description of the algorithm is given across Algorithms 3 and 4. The decoder works in an iterative fashion - as already mentioned - where there is one main subroutine that is supposed

to recover a single solution if it exists. If a solution is found, all agreeing inputs A are removed from In' and we look for additional solutions. In the main subroutine, FindOne, we use the basic lattice decoder described earlier. When a solution is not found from the reduced basis, we first check if we can remove points from a temporary *working set* ws. If this isn't possible we use the subroutine FindTarget on what is left over to find a solution.

Proof of Theorem 4

Proof. We note that for correctness it suffices to show that FindOne recovers a solution if any exists corresponding to the current input, that Algorithm 4 does not terminate too early, and that no points are erroneously deleted. By Lemma 6 and the algorithm description, we only remove points that correspond to solutions and any removed point does not agree with any other solution. Algorithm 4 only terminates when FindOne returns $\perp$. FindOne only returns bot when the shortest vector in the lattice is too large, which cannot happen if there is still a target vector $\vec{v}$ that you should recover in $L(M)$. When running FindOne there are only two possibilities: either there is one shortest target vector that should be recovered or there are multiple. Lemma 7 tells us if there is only one it will appear at line 9 of FindOne. So, suppose there is more than one. Together lines 14-15 and Lemma 7 imply that the basis B that is used in FindTarget has the property that it equals $U\hat{S}$ where $\hat{S}$ is a matrix representing some subset of solution vectors and U has entries only in $\mathbb{F}$. By Lemma 8 we will recover one of the solution vectors with high probability. Taken all together, the algorithm recovers all close input codewords with high probability. □

Algorithm 4: Lattice-Based IRS Decoder

Input : $k, t, n, \mathsf{In} = \{(\alpha_i, \beta_{i,1}, \ldots, \beta_{i,c})\}_{i=1}^{n}$
Output: a (possibly empty) set of tuples $\{(p_1^i \ldots p_c^i)\}_{i=1}^{\ell}$

1 $solns := []$
2 $\mathsf{In}' := \mathsf{In}$
3 continue := True
4 **while** continue **do**
5 $res \leftarrow \mathsf{FindOne}(k, t, n, \mathsf{In}')$
6 **if** $res = \perp$ **then**
7 **return** $solns$
8 **else**
9 $(g_1, \ldots, g_c) \leftarrow res$
10 Append to $solns$ $(g_1, \ldots, g_c)$
11 $A \leftarrow \{(\alpha, \beta_1, \ldots \beta_c) \in \mathsf{In}' : (g_1(\alpha), \ldots, g_c(\alpha)) = (\beta_1, \ldots, \beta_c)\}$
12 $\mathsf{In}' \leftarrow \mathsf{In}' \setminus A$
13 **endw**

5 Handling Low Degree Inputs

Unfortunately the algorithm of the previous section cannot handle the case where some of the target vectors include *low-degree* polynomials i.e. $\deg(p_{i,j}) < k$. In this setting, Lemma 7 no longer holds. There may be combinations of the target vectors which are *shorter* than the original target vectors when taken with respect to the "standard" shift $(k, 0, \ldots, 0)$.

A preliminary example Suppose a received word is equidistant to two codewords corresponding to polynomial sets of degree $(k-2, k-3)$. In this case, for the standard shift, the target vectors $\vec{t_1}, \vec{t_2}$ are unbalanced with degrees $[(n-t)+k, (n-t)+k-2, (n-t)+k-3]$. It is relatively easy to find two independent vectors shorter than these targets. $\vec{t_1} - \vec{t_2}$ is shorter since the first entry drops by one degree. Note that $z \cdot \vec{t_1} - z \cdot \vec{t_2}$ has the same degree as $\vec{t_1}$ and $\vec{t_2}$. Moreover, in $\vec{t_1} - \vec{t_2}$ the first position is the only position achieving the highest degree and it has some leading coefficient $v_0 \in \mathbb{F} \setminus \{0\}$. We know then that $z\vec{t_1} - (z + v_0)\vec{t_2}$ also has degree one less than the targets.

$$M = \begin{bmatrix} z & -(z+v_0) \\ 1 & -1 \end{bmatrix}$$

The determinant of the matrix M is clearly not 0, so the vectors we have found are independent. This is a rather limited example, but more interesting ones can be found if there are more close codewords and the degree patterns have a larger variety.

How can we still facilitate recovery with so little guarantees on the form of our inputs? Perhaps surprisingly, this is possible using a minor modification to our prior algorithm, provided we find a special reduced basis for this lattice with respect to a *correct* shift $\vec{s}$. The shift $\vec{s}$ will ensure that the basis B we find satisfies the property that $B = US$ where $U \in \mathbb{F}^{m \times m}$ or there will be a row in U with a zero entry. If finding this is possible, the previous strategy will still help us to recover a target vector.

The correct shift we consider is an $\vec{s}$ for which S is row-reduced. Recall we do not know S so it might be hard to imagine how we could find such a shift. For now, assume we can. As we definitely have the basis B in $\vec{s}$ reduced weak popov form then by Theorem 2, $\deg(U)_{i,j} \leq \vec{\tau}_i - \vec{\gamma}_j \leq \text{perm}(\gamma)_i - \vec{\gamma}_j$. This inequality immediately tells us that either U has some zero entry or $\gamma = (d, \ldots, d)$ for some constant $d \in \mathbb{Z}$ and U has all constant entries.

Below, we give some evidence that if the leading coefficient of each polynomial is chosen uniformly at random, and the field size $\mathbb{F}$ is large, it should be possible to find this shift.

5.1 Existence of a good shift

The key goal of this section will be to show that there exists a shift $\vec{s}$ which makes a permutation of entries in $LC(Sz^{\vec{s}})$ non-zero. Since each entry in S is

chosen uniformly at random, with high probability S will end up being full rank when we hit this condition. Below, we first show that this $\vec{s}$ exists. We then give an algorithm which appears to find this good shift and some intuition for why it might work the way it does.

Definition 9 (Traversal Pattern)**.** *A traversal pattern $P \in ([n] \times [m])^n$ where $n < m$ has the property that if $(i,j) \in P$ then $\forall j' \neq j, (i,j) \notin P$ and $\forall i' \neq i, (i,j) \notin P$. In words, P is a permutation in the first coordinate with no repeated entries in the second.*

Definition 10 (Traversal Path)**.** *A (traversal) path $P \in ([n] \times [m])^{<n}$ where $n < m$ has the property that if $(i,j) \in P$ then $\forall j' \neq j, (i,j) \notin P$ and $\forall i' \neq i, (i,j) \notin P$. In words, P has no repeating x or y coordinates.*

Theorem 11 (Existence of a good shift)**.** *Given $S \in \mathbb{F}[z]^{n \times m}$ where $n < m$ and $\forall i \in [n], j \in [m], S_{i,j} \neq 0$, $\exists$ a shift $\vec{s}$ and traversal pattern $P \in ([n] \times [m])^n$ on $Sz^{\vec{s}}$ so that $\forall (i,j) \in P, LC(Sz^{\vec{s}})_{i,j} \neq 0$*

Proof. Define $D = (d_{i,j})$ where $d_{i,j} = \deg(S_{i,j})$ to be the degree matrix associated with S. Consider a modified version of D, called D' where D' is obtained by taking each row i of D, calculating the maximum entry d_i^*, and subtracting it from every other $d_{i,j}$ in the row. Note that every row has a 0. We will call this the essential form of D. We can increase a column j of D/D' by some constant c so that $\forall j \in [n], d_{i,j} = d_{i,j} + c$. For the essential form, if $\exists i'$ s.t. $d_{i',j} + c > 0$ the we say entry (i', j) has *flipped*, and we must *re-weight* row i', subtracting every entry by $d_{i',j} + c$.

Note that the operation of increasing each entry of D' in column j by $v \in \mathbb{Z}^+$ tracks what occurs to the degree matrix when column j of S is shifted by z^v. Moreover, if D' has a *traversal pattern* P made up solely of 0 entries, then P will also be a traversal pattern for S for which there exists an $\vec{s}$ fulfilling the condition. Therefore the condition above is equivalent to finding updates to D' that result in 0 entries for at least one traversal pattern.

Consider the following algorithm which iteratively finds such a P. Let $\mathcal{P}$ be the set of all possible traversal paths that can be formed by only entries (i,j) where $d'_{i,j} = 0$. Order the paths in $\mathcal{P}$ by length, and let $d^* < n$ denote the length of the maximum length path in $\mathcal{P}$. Let P be one of those paths of that length. Choose some $(i,j) \in [n] \times [m]$ s.t. $P \cup \{(i,j)\}$ makes a traversal path of length $d^* + 1$. Our goal will be to update D' so that the new entry is strictly greater than $d_{i,j}$. If the new entry is 0 we have extended the path but even if it's not, if we can argue it is always getting closer to 0 it must reach 0 eventually.

First, we increase column j by $d_{i,j}$ which sets $d_{i,j}$ to 0. After re-weighting, we consider the list of all $(i', j') \in P$. If $d_{i',j'}$ is no longer 0, we update column j' to add by $-d_{i',j'}$ we do this until all entries in P are 0.

It will never be the case that any column j that shows up in an entry of P needs to be shifted up more than once. Every $(i', j') \in P$ only needs to be updated if re-weighting happened in row i' because of an earlier flip in another column j'' by some value v. This re-weighting must have effected every entry

in row i' except for (i', j''). For (i', j') to flip again, either increasing another column must lead to another re-weighting in row i' or we must increase column j'. Since only one entry per column is allowed in P there is no reason to increase column j' again. Because the value used in an earlier re-weighting can only stay the same or decrease, any later update to a column $j''' \neq j''$ will not be large enough to flip the entry in (i', j''') and because we already flipped (i', j''), we will not flip it again.

Note that the update algorithm allows for $d_{i,j} \neq 0$ at the end. This is due to "cycles" in the updates which start with (i, j) and end at (i, j). As we said earlier, if $d_{i,j}$ is updated to a larger value, we are done. It is never updated to a smaller value: even when there are multiple cycles that effect row i, the overall change in entry (i, j) is only by the maximum value that is added to one of the columns, which will leave us with something at least as large as $d_{i,j}$. Suppose $d_{i,j}$ stays the same after the update. Then for at least one cycle, every column update had a value of $-d_{i,j}$. For that cycle, consider the update path which contains entries in P and (i, j) values that flipped, implicating more entries in P. Let the cycle contain z nodes in P. Then all the other nodes could form a path P^* of length $z + 1$ because they all must have had a value of 0 and all the indices are distinct. The only reason that we would not have used the nodes in P^* to form our longest path is if there are nodes in P which have non-distinct columns and rows from P^*. But if that were true, then the entries in P along the update path would also be in conflict with this entry, because those in P^* always share a column and a row with some entry in P. Thus, it isn't possible for a cycle to end without increasing the entry $d_{i,j}$ in D'. This ends the proof. □

Of course, the above is of no use to us when devising an algorithm. Just because a shift may exist does not mean we can efficiently find it. In Algorithm 5 we give one such iterative procedure. In words, we start with the standard shift and attempt to find a reduced form of B. We then check the reduced form's pivot entries I_{piv}. If the error locator position is not in the set, we terminate with the given shift. If it *is*, we update all the other entries that are not in the pivot set, raising them by 1. The intuition is that because some polynomials are too small, many entries of $LC(S)$ will be 0 which could lend to the error locator column being in the pivot. We shift up the positions not in the pivot because it is likely that they come from columns where the degree of all entries is too low, so these values are not imposing constraints that should be imposed. Heuristically, this algorithm seems to work quite well for finding a good $\vec{s}$.

Putting it all together The previous algorithm is meant to be used *before* FindTarget to ensure that FindTarget is successful. The full algorithm that will also work with random polynomials with degree less than k is the same as the previous one except we use FindOne with the gray box and without line 19.

6 Notes on some failure cases and workarounds

As we said previously, the algorithm above is heuristic. In this portion we expand on a few clear failure cases and methods that can be used to deal with them.

Small fields. When the size of $\mathbb{F}$ is small, i.e. $|\mathbb{F}| < 2^{10}$, it sometimes happens that Assumption 5 is violated. In particular, for our short basis B and the vector of target vectors S, $B = US$ where U is *fractional* e.g. $U = \frac{1}{z-\alpha}U'$ where U' is unimodular. One method that seems to help with this problem is by increasing the multiplicity and shift. That is, the current algorithm comes from taking the lattice formed by [4] and using the multiplicity and shift values of 1, then looking at it's scaled dual. If we take higher shift and multiplicity, we get larger matrices with more constraints. This lattice is almost the same as one which would get from extending the power decoder of [9] to the interleaved setting.

When there is not independence. Many of the previous statements relied on the polynomials p having coefficients that were chosen uniformly at random. Even when just the leading coefficients are chosen so that one is in the span of the others, the assumption is violated. In practice, we cannot find a shift $\vec{s}$ that results in a good matrix U which has all 0 entries. However, Algorithm FindTarget can sometimes output a W which can be searched for points to temporarily erase, in a way that is analogous to lines 14-15 of Algorithm 3.

Algorithm 5: FindShift

Input : k and a short basis $B \in \mathbb{F}[z]^{m\times(c+1)}$
Output: a shift $\vec{s} \in \mathbb{Z}^{c+1}$

1 $\vec{s} := (k, 0, \ldots, 0)$
2 $\mathsf{In}' := \mathsf{In}$
3 notFound := True
4 **while** notFound **do**
5 $\quad B', I_{\mathsf{piv}} \leftarrow \mathsf{WeakPopovWithPivot}(B, \vec{s})$
6 $\quad$ **if** $0 \notin I_{\mathsf{piv}}$ **then**
7 $\quad\quad$ notFound = False
8 $\quad$ **else**
9 $\quad\quad$ **for** $i \in [c]$ **do**
10 $\quad\quad\quad$ **if** $i \notin I_{\mathsf{piv}}$ **then**
11 $\quad\quad\quad\quad s'_i \leftarrow s_i + 1$
12 $\quad\quad\quad$ **else**
13 $\quad\quad\quad\quad s'_i \leftarrow s_i$
14 $\quad\quad$ **endfor**
15 $\quad\quad \vec{s} \leftarrow (s'_0, s'_1, \ldots, s'_c)$
16 **endw**
17 **return** $\vec{s}$

Related Work In [6] the authors used a variant of the algorithm presented here, unknowingly for the specific setting where the polynomials are random *monomials* of degree k. The astonishing result of [3] also recently showed that list-decoding of any reed solomon code of constant rate can be done deterministically in polynomial time. This obviously covers the interleaved setting as well, although the techniques used in that work are for the dual formulation of the one proposed here.

Acknowledgements The majority of this work was done while the author was at the University of Montpellier working as a CNRS researcher. They were funded by a grant from l'Institut Cybersécurité Occitanie (ICO). The beginning quarter was done while the author was at Johns Hopkins University. The author would like to thank Romain Lebreton, Eleonora Guerrini, Nicole Franzoi, and Vincent Neiger for various discussions about the work in question. All mistakes are the author's. No AI was used to come up with the ideas presented in this document nor in its writing.

References

[1] Bernhard Beckermann, George Labahn, and Gilles Villard. Shifted normal forms of polynomial matrices. In *Proceedings of the 1999 international symposium on Symbolic and algebraic computation*, pages 189–196, 1999.

[2] Daniel Bleichenbacher, Aggelos Kiayias, and Moti Yung. Decoding of interleaved reed solomon codes over noisy data. In *International Colloquium on Automata, Languages, and Programming*, pages 97–108. Springer, 2003.

[3] Joshua Brakensiek, Yeyuan Chen, Aaron (Louie) Putterman, Zihan Zhang, and Kai Zhe Zheng. Algorithmic list decoding of reed-solomon codes up to capacity in the low rate regime.

[4] Henry Cohn and Nadia Heninger. Approximate common divisors via lattices, 2012.

[5] Henry Cohn and Nadia Heninger. Ideal forms of coppersmith's theorem and guruswami-sudan list decoding, 2013.

[6] Harry Eldridge, Gabrielle Beck, Matthew Green, Nadia Heninger, and Abhishek Jain. {Abuse-Resistant} location tracking: Balancing privacy and safety in the offline finding ecosystem. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5431–5448, 2024.

[7] Thom Mulders and Arne Storjohann. On lattice reduction for polynomial matrices. *Journal of symbolic computation*, 35(4):377–401, 2003.

[8] Vincent Neiger, Johan Rosenkilde, and Grigory Solomatov. Computing popov and hermite forms of rectangular polynomial matrices. In *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*, pages 295–302, 2018.

[9] Johan Rosenkilde. Power decoding reed–solomon codes up to the johnson radius. *arXiv preprint arXiv:1505.02111*, 2015.

Algorithm 3: FindOne, the main subroutine for the decoder over Ch'

Input : $k, t, n, \mathsf{In} = \{(\alpha_i, \beta_{i,1}, \ldots, \beta_{i,c})\}_{i=1}^{n}$
Output: $(p_1, \ldots, p_c) \in \mathbb{F}[z]^c$ or $\perp$

1 $ws := [n]$
2 continue := True
3 **while** continue **do**
4 $\quad \forall j \in [c], f_j(z) = \mathsf{LagrInterpol}(\{(\alpha_i, \beta_{i,j})\}_{i \in [ws]})$, $N(z) = \prod_{i \in [ws]}(z - \alpha_i)$
5

$$M = \begin{bmatrix} 1 & f_1(z) & f_2(z) & \ldots & f_c(z) \\ & N(z) & & & \\ & & N(z) & & \\ & & & \ddots & \\ & & & & N(z) \end{bmatrix}$$

6 $\quad M_{\mathsf{red}} \leftarrow \mathsf{WeakPopov}(M, (k, 0, \ldots, 0))$
7 $\quad$ **if** $\lambda_1(M_{\mathsf{red}}) > k + (|ws| - t)$ **then**
8 $\quad\quad$ **return** $\perp$
9 $\quad$ Look for a vector $\vec{v}$ of form $(z^k e(z), p_1(z)e(z), \ldots, p_c(z)e(z))$ in M_{red}
10 $\quad$ **if** $\vec{v} \neq 0$ **then**
11 $\quad\quad$ **return** $(p_1, \ldots, p_c)$
12 $\quad$ **else**
13 $\quad\quad$ $W = \{\alpha \in \mathbb{F} : (z - \alpha) | N(z), \exists \vec{r}$ a row vector in M_{red} and $(z - \alpha) | \vec{r}\}$
14 $\quad\quad$ **if** $W \neq \emptyset$ **then**
15 $\quad\quad\quad$ $ws \leftarrow ws \setminus W$
16 $\quad\quad$ **else**
17 $\quad\quad\quad$ Construct $B \in \mathbb{F}[z]^{m \times (c+1)}$ from row vectors $\vec{r}$ in M_{red} s.t. $\deg(\vec{r}) \leq k + (|ws| - t)$

$\quad\quad\quad$ $\vec{s} \leftarrow \mathsf{FindShift}(k, B)$
$\quad\quad\quad$ $B' \leftarrow \mathsf{WeakPopov}(B, \vec{s})$
$\quad\quad\quad$ $W' = \{\alpha \in \mathbb{F} : (z - \alpha) | N(z), \exists \vec{r}$ in B' and $(z - \alpha) | \vec{r}\}$
$\quad\quad\quad$ **if** $W' \neq \emptyset$ **then**
$\quad\quad\quad\quad$ $ws \leftarrow ws \setminus W'$
$\quad\quad\quad$ **else**
$\quad\quad\quad\quad$ **return** $\mathsf{FindTarget}(|ws|, k, t, B', \{(\alpha_i, \beta_{i.1}, \ldots, \beta_{i,c})\}_{i \in ws})$

18
19 $\quad\quad\quad$ **return** $\mathsf{FindTarget}(|ws|, k, t, B, \{(\alpha_i, \beta_{i.1}, \ldots, \beta_{i,c})\}_{i \in ws})$
20 **endw**

Figure 1: Depicts two variants of the FindOne algorithm. In the first variant the lines in the gray box are not executed. In the second, they are executed in place of Line 19.